

串口通信协议文档

概述

本文档描述了一个基于串口的双向通信协议，支持设备（客户端）与主机（服务端）之间的数据传输、握手机制和消息确认。

- 客户端：嵌入式设备端（本代码库的实现）
- 服务端：主机端（如 Python 测试脚本）

协议格式

消息结构

所有消息采用统一格式，按以下顺序组成：

字段	长度	说明	示例值
同步头	1 字节	固定值 0xA5	0xA5
用户ID	1 字节	用户标识，固定为 0x01	0x01
消息类型	1 字节	消息类型标识	0x01, 0x04, 0x05, 0x06, 0xFF
消息长度	2 字节	数据部分长度（小端模式）	0x04 0x00 (4字节)
消息ID	2 字节	消息序列号（小端模式）	0x01 0x00 (ID=1)
消息数据	N 字节	实际数据内容	可变
校验码	1 字节	校验和（补码）	计算得出

总长度 = 7（头部）+ N（数据长度）+ 1（校验码）= 8 + N 字节

字节序

- 消息长度：小端模式（低字节在前）
- 消息ID：小端模式（低字节在前）

校验码计算

校验码 = (~所有字节之和 + 1) & 0xFF

计算范围：从同步头到消息数据结束，不包括校验码本身。

示例：

```
sum = 0
for byte in message[:-1]: # 不包括最后的校验码
    sum += byte
checksum = ((~sum) + 1) & 0xFF
```

消息类型定义

客户端发送的消息

0x01 - 握手消息

用途：客户端主动发起握手，确认与服务端的连接。

数据格式:

```
0xA5 0x00 0x00 0x00
```

完整消息示例（十六进制）：

[illegible]

发送时机:

- 客户端启动后立即开始发送
- 每 500ms 发送一次
- 收到服务端的握手确认 (0xFF) 后停止

0x04 - 设备消息/唤醒消息

用途：客户端向服务端报告设备状态、唤醒事件等。

数据格式: JSON 字符串 (UTF-8 编码)

示例场景:

1. 唤醒事件

```
{
  "type": "aiui_event",
  "content": {
    "beam": 0,
    "angle": 45,
    "confidence": 950
  }
}
```

0x05 - 用户数据消息（客户端→服务端）

用途：客户端向服务端传输业务数据。

数据格式: JSON 字符串 (UTF-8 编码) 或二进制数据

示例场景:

2. 启动版本信息（客户端初始化时自动发送）

```
{
  "code": 0,
  "content": "version: circle6:v1.3.0 engine:se_circle.1004; build date: Thu Jun 16 14:29:55 CST 2025",
  "type": "started"
}
```

3. 其他业务数据：根据具体应用场景定义

0x06 - 音频数据消息

用途：客户端向服务端传输音频数据流。

数据格式：原始音频数据（PCM 格式）

说明：音频数据可以分包发送，服务端需按消息ID顺序接收并拼接。

服务端发送的消息

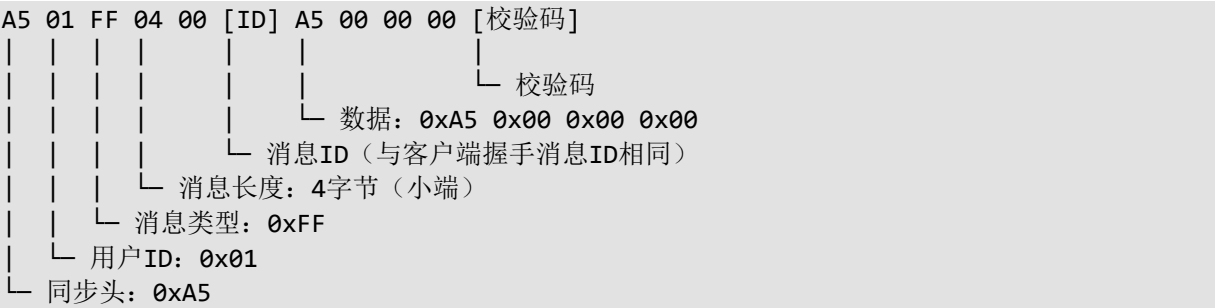
0xFF - 握手确认消息

用途：服务端响应客户端的握手请求。

数据格式：

```
0xA5 0x00 0x00 0x00
```

完整消息示例（十六进制）：



重要：

- 消息ID必须与收到的握手消息（0x01）的ID相同
- 客户端通过匹配消息ID来确认握手成功

0x05 - 用户数据消息（服务端→客户端）

用途：服务端向客户端发送控制指令或查询请求。

数据格式：JSON 字符串（UTF-8 编码）

示例场景：

1. 手动唤醒

```
{
  "type": "manual_wakeup",
  "content": {
```

```
    "beam": 0
  }
}
```

2. 获取原始音频

```
{
  "type": "get_original_audio",
  "content": {
    "audio": 1
  }
}
```

3. 设置唤醒关键词

```
{
  "type": "wakeup_keywords",
  "content": {
    "keyword": "小爱同学",
    "threshold": "900"
  }
}
```

4. 获取版本信息

```
{
  "type": "version"
}
```

客户端确认消息

客户端收到服务端的控制指令后，会处理该指令并发送确认消息。

0x05 - 确认消息（客户端→服务端）

用途：客户端向服务端反馈指令执行结果。

数据格式：JSON 字符串（UTF-8 编码）

消息结构：

```
{
  "code": 0,
  "content": "success"
}
```

字段	类型	说明
code	整数	执行结果码：0 表示成功，非 0 表示失败
content	字符串	结果描述信息（成功消息或错误详情）

重要：

- 消息ID必须与收到的控制指令消息ID相同
- 服务端通过匹配消息ID来确认是哪个指令的响应

确认消息示例

1. 手动唤醒成功

```
{
  "code": 0,
  "content": "success"
}
```

2. 设置波束失败

```
{
  "code": -1,
  "content": "Invalid beam value: 10, valid range is 0-5"
}
```

3. 唤醒关键词设置成功

```
{
  "code": 0,
  "content": "success"
}
```

4. JSON 解析失败

```
{
  "code": -1,
  "content": "Invalid JSON message"
}
```

5. 版本查询成功

```
{
  "code": 0,
  "content": "版本号"
}
```

错误码说明

错误码	说明
0	执行成功
-1	通用错误（具体错误信息见 content 字段）

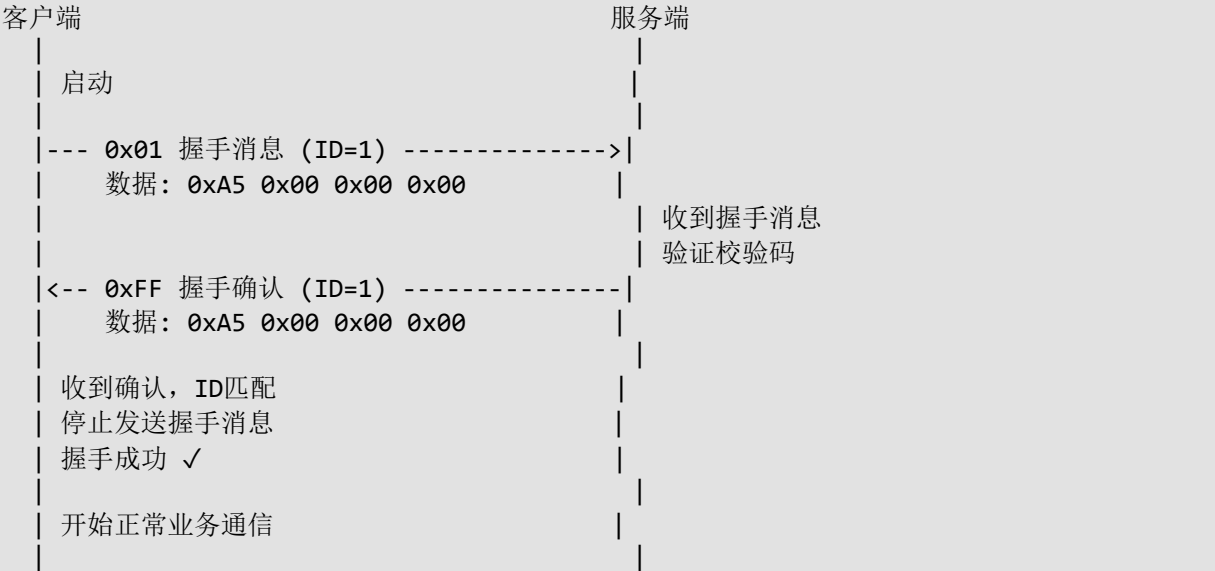
常见错误信息

错误内容	原因
Invalid message type	消息类型字段缺失或无法识别
Invalid JSON message	JSON 格式错误，无法解析
Invalid message: beam field must be a number	波束参数类型错误
Invalid beam value: X, valid range is 0-5	波束值超出有效范围
Failed to set beam: X	设置波束失败（底层错误）
Invalid message: keyword field must be a string	关键词参数类型错误
Invalid message: threshold value is out of range	阈值超出有效范围
Failed to add wake word	添加唤醒词失败

Invalid message: audio field must be a number	音频标志参数类型错误
---	------------

通信流程

1. 握手流程



2. 数据通信流程



3. 完整请求-响应示例

示例 1：设置波束（成功）

服务端 -> 客户端（消息类型：0x05，ID：10）

```
{
  "type": "manual_wakeup",
  "content": {
    "beam": 2
  }
}
```

客户端处理：

1. 解析 JSON
2. 提取 beam 值：2
3. 验证范围：0-5 ✓
4. 调用底层接口设置波束
5. 设置成功

客户端 -> 服务端（消息类型：0x05，ID：10，相同ID）

```
{
  "code": 0,
  "content": "success"
}
```

示例 2：设置波束（失败）

服务端 -> 客户端（消息类型：0x05，ID：11）

```
{
  "type": "manual_wakeup",
  "content": {
    "beam": 10
  }
}
```

客户端处理：

1. 解析 JSON
2. 提取 beam 值：10
3. 验证范围：0-5 X（超出范围）
4. 返回错误

客户端 -> 服务端（消息类型：0x05，ID：11，相同ID）

```
{
  "code": -1,
  "content": "Invalid beam value: 10, valid range is 0-5"
}
```

示例 3：设置唤醒关键词

服务端 -> 客户端（消息类型：0x05，ID：12）

```
{
  "type": "wakeup_keywords",
  "content": {
    "keyword": "小爱同学",
    "threshold": "900"
  }
}
```

```
客户端处理：
1. 解析 JSON
2. 提取 keyword: "小爱同学"
3. 提取 threshold: 900
4. 验证 threshold 范围: 500-1500 ✓
5. 生成唤醒词配置
6. 设置成功

客户端 -> 服务端 (消息类型: 0x05, ID: 12, 相同ID)
{
  "code": 0,
  "content": "success"
}
```

示例 4：获取版本信息

```
服务端 -> 客户端 (消息类型: 0x05, ID: 13)
{
  "type": "version"
}

客户端处理：
1. 识别类型为 version
2. 获取版本字符串

客户端 -> 服务端 (消息类型: 0x05, ID: 13, 相同ID)
{
  "code": 0,
  "content": "版本号"
}
```

串口配置参数

默认配置

参数	值
设备路径	/dev/ttyGS0
波特率	115200
数据位	8
校验位	无
停止位	1
流控制	无

支持的波特率

- 115200

消息ID管理

客户端

- 消息ID从 0 开始，自动递增
- 每次发送新消息时递增（0-65535，循环）
- 握手消息也占用消息ID

服务端

- 维护独立的消息ID计数器
- 握手确认消息使用客户端握手消息的ID（不占用自己的ID）
- 其他消息使用自己的ID计数器

协议限制

项目	限制
最大消息长度	10240 字节（包括头部和校验码）
消息ID范围	0-65535（循环使用）
最大数据长度	10232 字节（10240 - 8）
握手超时	无限重试，直到收到确认

错误处理

校验码错误

- 接收方必须验证校验码
- 校验失败的消息应直接丢弃
- 不发送错误响应

消息格式错误

- 同步头不匹配：继续查找下一个同步头
- 用户ID不匹配：丢弃消息
- 消息长度超限：丢弃消息

握手失败

- 客户端：持续发送握手消息，不超时
- 服务端：静默等待握手消息

缓冲区溢出

- 非阻塞写入，缓冲区满时等待最多 10ms
- 仍然无法写入时，部分数据可能丢失

注意事项

1. **握手机制**: 必须先完成握手才能进行正常通信
2. **消息ID匹配**: 握手确认消息的ID必须与握手消息的ID相同
3. **校验码验证**: 所有消息都必须验证校验码
4. **消息长度**: 确保消息总长度不超过 10240 字节
5. **非阻塞模式**: 建议使用非阻塞I/O, 避免阻塞通信线程
6. **字节序**: 消息长度和消息ID使用小端模式
7. **版本信息**: 客户端启动时自动发送版本信息, 握手前后均可发送

测试工具

项目提供了 Python 测试脚本 `test/serial_reader.py`, 支持:

- 自动响应握手消息
- 解析和验证所有消息类型
- 发送控制指令
- 保存音频数据到 PCM 文件

使用方法:

```
python3 test/serial_reader.py -p /dev/ttyACM0 -b 115200
```